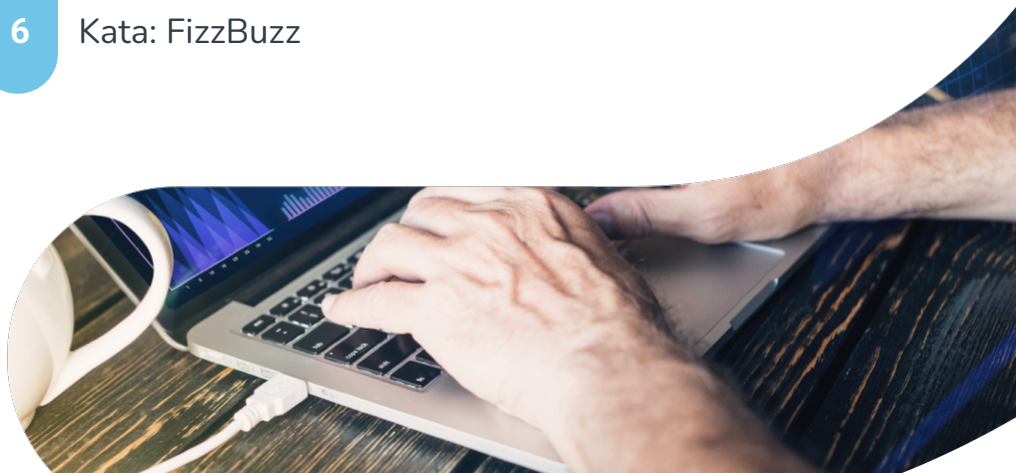
Test Driven Development: ¡tu código sostenible y de calidad!



Ana María García Sánchez

Índice
Parte 1

1. ¿De dónde venimos?
2. ¿Por qué testear?
3. ¿Cómo debe ser un test?
4. TDD: ¿Qué es?
5. Midiendo la calidad del código
6. Kata: FizzBuzz



Índice
Parte 2

7 JUnit
8 SonarQube
9 Kata: tenis
10 Mockito
11 Cucumber y Testcontainers
12 Entregable: Kata Store



01_

¿De dónde venimos?



+140 especialistas

ÁREAS

- Cloud
- Data & AI
- Analítica Avanzada
- Desarrollo de Software

SECTORES

- TRANSPORTE
- LOGÍSTICA
- TELECOMUNICACIONES
- SEGUROS
- CONSTRUCCIÓN
- ENERGÍA








Azure amazon web services dbt

sirviendo código;

SirviendoCódigo.com

www.youtube.com/@SirviendoCódigo





Ana García

Software Engineer en Next Digital

- Enamorada del **código limpio** con **más de 6 años en el sector**
- **Speaker** internacional
- Enamorada aún más de sus gatas
- Hoy imparte un taller sobre **TDD**

ana.garcia@nextdigital.es

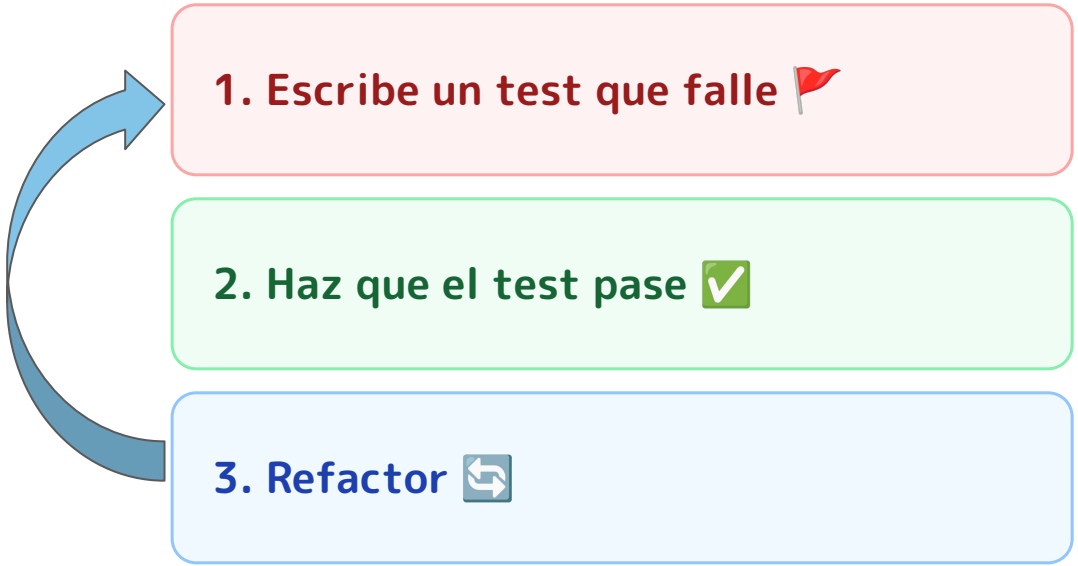
linkedin.com/in/anagciaschz/es

@AnaGciaSchz

next digital

Spoiler: ¿Qué es el TDD?

Metodología de desarrollo guiado por pruebas.



02_

¿Por qué testear?



¿Por qué testear?

RAZÓN #1

Por ti mismo

Ya lo estás haciendo, pero a mano cada vez que ejecutas tu aplicación.

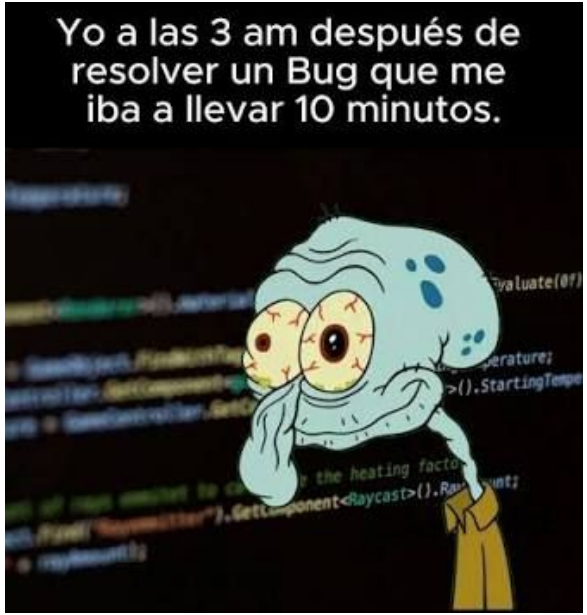


¿Por qué testear?

RAZÓN #2

Asegurar la robustez de software

- Detectamos errores antes de que lleguen a producción
- Garantizamos que el software funciona según lo esperado



¿Por qué testear?

RAZÓN #3

Mantenimiento de código

- Los tests son un “seguro”, puedes realizar cambios con **confianza**
- Fomenta un diseño más limpio



¿Por qué testear?

RAZÓN #4

Edge-cases

- Pensamos ***edge-cases*** que no se nos ocurrirían de otra manera
- Casos que son difíciles de recrear manualmente

When the Tester finds a critical bug at deployment time



¿Por qué testear?

RAZÓN #5.1

VALOR INTERNO

Confianza en el equipo

- Podemos **refactorizar con más libertad**, para mantener el código limpio.
    - **Sin miedo** a introducir nuevo código o funcionalidades
- Promueve cultura de **responsabilidad compartida** por la calidad
- Facilita **iteraciones más rápidas** y frecuentes en proyectos ágiles

RAZÓN #5.2

VALOR EXTERNO

Producto y Cliente

- ***Demos*** de producto más robustas
    - A priori no deberían detectar errores
- Confianza para los **usuarios y el cliente**

¿Por qué testear? Coste y tiempo ⏳💸

RAZÓN #6

IMPACTO EN PRODUCCIÓN

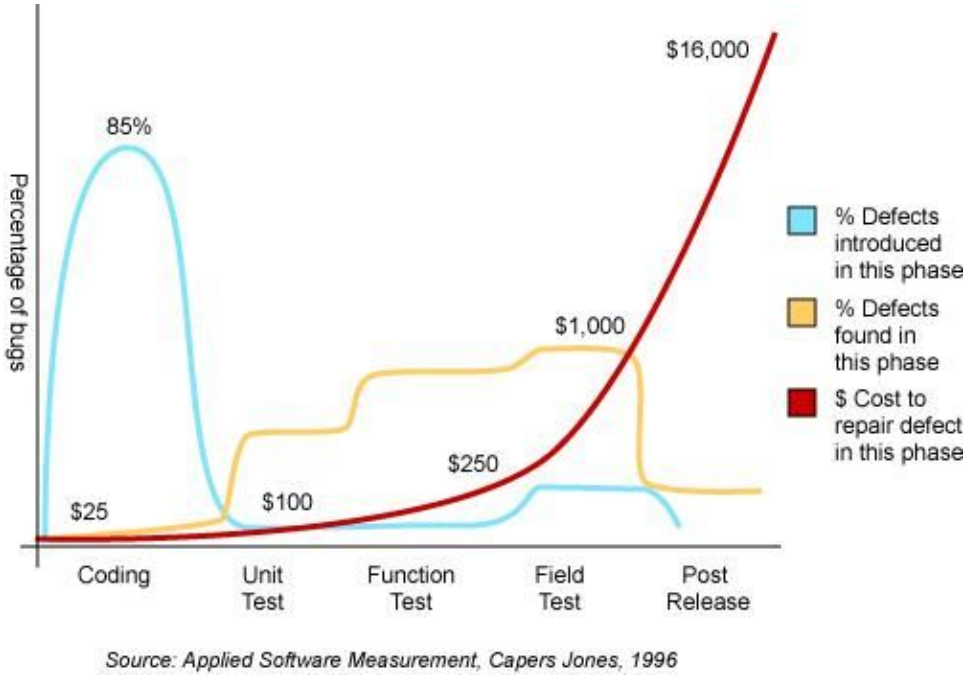
Mayor coste y tiempo

El coste y tiempo de resolución de un bug es exponencialmente mayor en producción.

OBJETIVO PRINCIPAL

Detección temprana

Detectar los bugs cuanto antes para ahorrar tiempo y costes a largo plazo.



Source: Applied Software Measurement, Capers Jones, 1996

¿Por qué testear? Pirámide del testing

RAZÓN #7

1. Unitarios

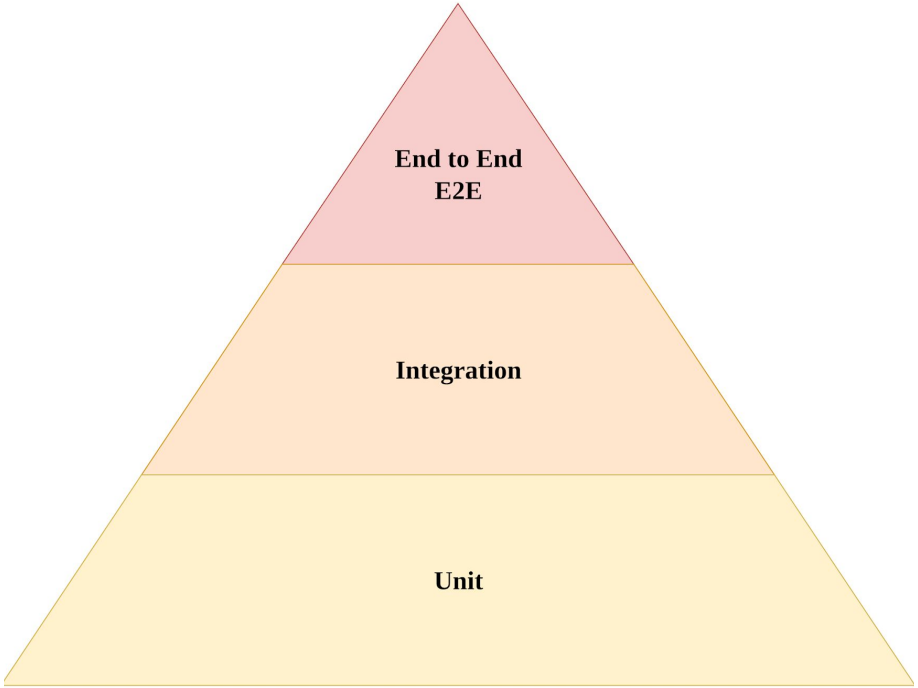
- En gran cantidad
- Rápidos en programar y ejecutar

2. Integración

- En menor cantidad
- Más lentos de programar y ejecutar

3. E2E

- Muy pocos evaluando caminos críticos
- Suelen costar mucho tiempo de programación y ejecución



Tipos de pruebas: el mapa completo

POR ALCANCE
Cuánto sistema prueban
Unitarias
Integración
Componente
Contrato
Sistema / end-to-end
Aceptación

FUNCIONALES
Qué hace el sistema
Humo (smoke)
Regresión
Exploratorias
Aceptación de usuario (UAT)

NO FUNCIONALES
Cómo lo hace
Rendimiento y carga
Estrés
Seguridad
Usabilidad
Accesibilidad
Compatibilidad

En negrita, las que vemos en el taller. La pirámide ordena las primeras por cantidad y coste.

Tipos de pruebas que vemos en el taller

UNITARIAS

Prueban una clase o función aislada. Las dependencias se sustituyen por dobles.

- Milisegundos
- Muchas

EN EL TALLER

JUnit + Mockito

INTEGRACIÓN

Prueban varios componentes reales juntos: base de datos, Spring, HTTP.

- Segundos
- Menos que las unitarias

EN EL TALLER

Testcontainers

E2E

Comprueban un requisito desde fuera, cómo lo usaría el cliente, sin mirar el código y recorriendo el sistema completo.

- Las más lentas
- Pocas, en los caminos críticos

EN EL TALLER

Cucumber



03_

¿Cómo debe ser un test?

Principios FIRST

F — **Fast (rápido)**
Para ejecutar muchos en poco tiempo

I — **Independent (independiente)**
Los tests no pueden depender unos de otros

R — **Repeatable (repetible)**
Mismo resultado, independiente del entorno

S — **Self-validating (auto evaluable)**
Resultado claro: ok/ko

T — **Timely (oportuno)**
Deben escribirse idealmente antes de codificar y siempre antes de subir a producción

Patrón AAA

A

Arrange

Configura el **entorno** necesario para ejecutar la prueba: objetos requeridos, **estado inicial** de datos y define dependencias.

A

Act

Evento principal a probar.

A

Assert

Verifica que el **resultado de la acción** coincide con el comportamiento esperado.

Patrón AAA: Ejemplo

SchoolTest.java

```java
@Test  new *
void shouldReturnDirectionWhenSchoolExists() {
    // Arrange
    School school = new School();

    // Act
    String direction = school.getDirection();

    // Assert
    assertEquals( expected: "Oviedo, Asturias", direction);
}
```

Arrange

Configura el entorno y datos iniciales de la prueba.

Act

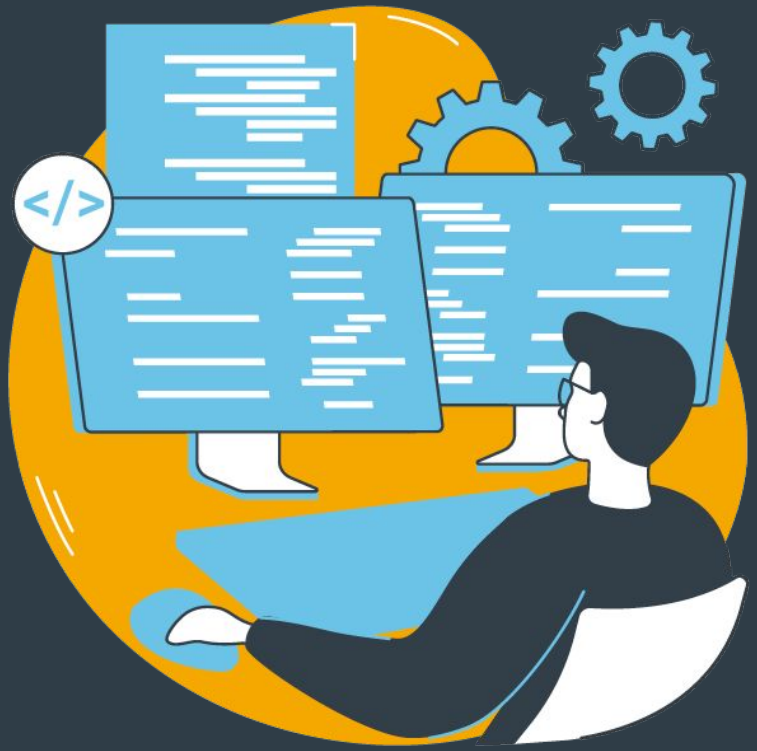
Ejecuta la acción o método principal a evaluar.

Assert

Verifica que el resultado coincide con lo esperado.

04_

TDD: ¿Qué es?



TDD: ¿Qué es?

Metodología de desarrollo guiado por pruebas.

Ciclo **Red**-**Green**-**Refactor**:

1. Escribe un test que falle 🚩

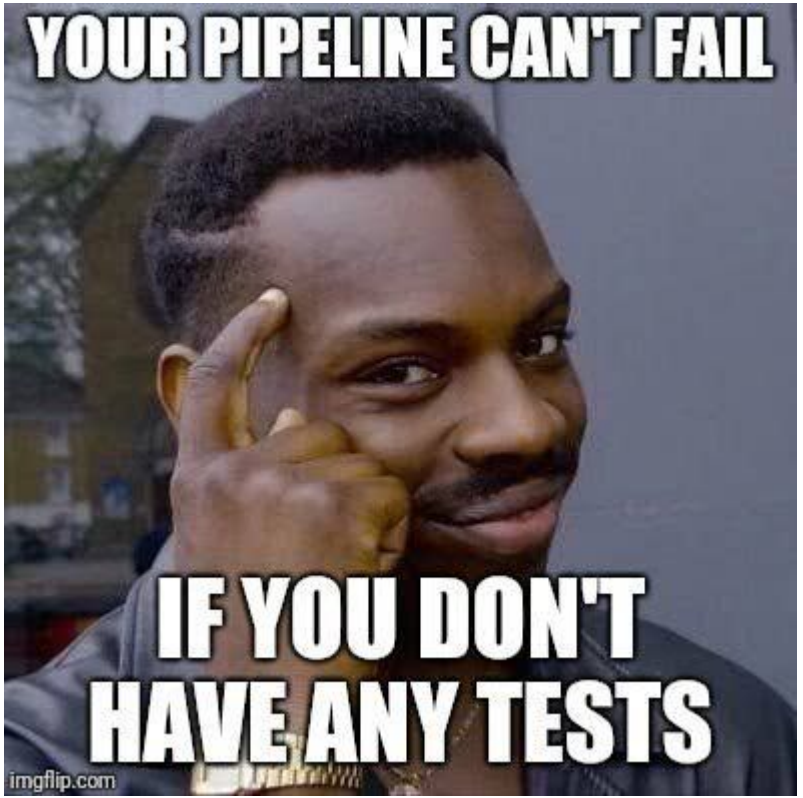
Define la funcionalidad deseada antes de escribir código.

2. Haz que el test pase ✅

Escribe el código mínimo necesario para cumplir la prueba.

3. Refactor 🔄

Mejora y limpia el código sin alterar el comportamiento.



TDD: ¿Qué es?

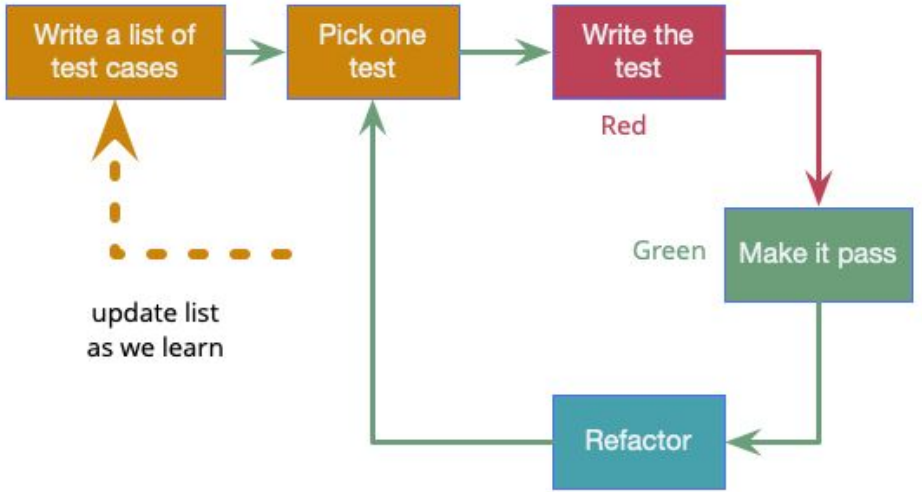
Importantísimo:

1. Identificar Test cases

Definir los escenarios antes de comenzar a codificar.

2. Refinar

Iterar y simplificar las listas de prueba continuamente.



Fuente: https://martinfowler.com/bliki/TestDrivenDevelopment.html

TDD: Beneficios

1. Diseño Desacoplado 🧩

Usar los tests al comienzo del desarrollo nos ayuda a implementar una arquitectura modular y desacoplada.

2. Código Más Limpio 🧼

Mantiene la base de código ordenada y legible. El refactor constante es parte integral del proceso.

3. Alta Cobertura de Test 🎯

La cobertura resulta de forma natural al aplicar la metodología, no como una meta impuesta.

4. Documentación Ejecutable 📖

Los tests explican y demuestran exactamente cómo debería comportarse el código en cada escenario.

TDD: Dificultades

DIFICULTADES

- Cuesta pensar el test antes que el código
- Al principio se va más lento
- Con requisitos poco claros no sabes qué test escribir
- En código heredado sin tests es difícil empezar
- Los tests acoplados a la implementación se rompen con cada cambio

CÓMO MITIGARLAS

- Practicar con katas pequeñas
- Pasos pequeños: un test, un comportamiento
- Aclarar los requisitos antes de escribir el primer test
- En código heredado, empezar con tests que fijen el comportamiento actual
- Probar comportamiento, no detalles de implementación

TDD: Regla Fundamental

REGLA DE ORO DE TDD

📣 **Solo escribimos código cuando tenemos un test que falle**

1. Escribir Test 🔴

Crear una prueba que falle para definir claramente el comportamiento esperado antes de programar.

2. Implementar 🟢

Escribir la cantidad mínima de código necesaria para lograr que el test pase exitosamente.

3. Refactorizar 🔵

Limpiar y optimizar la solución manteniendo todas las pruebas funcionando en verde.

Refactor: mejorar sin romper

REGLAS

- Solo se refactoriza con todos los tests en verde
- Pasos pequeños, ejecutando los tests tras cada uno
- El comportamiento no cambia: si hace falta uno nuevo, primero su test
- Refactor y nueva funcionalidad, nunca a la vez

QUÉ BUSCAR (CODE SMELLS)

- Código duplicado → extraer método
- Métodos largos → dividir
- Nombres poco claros → renombrar
- Condicionales anidados → retornos tempranos
- Números mágicos → constantes con nombre

05_

Midiendo la calidad del código



Calidad de código

Análisis de código

- **Complejidad ciclomática**
- **Cobertura de código**
- **Tamaño de método/clases** (LOC - Lines of code)
- **Código duplicado**

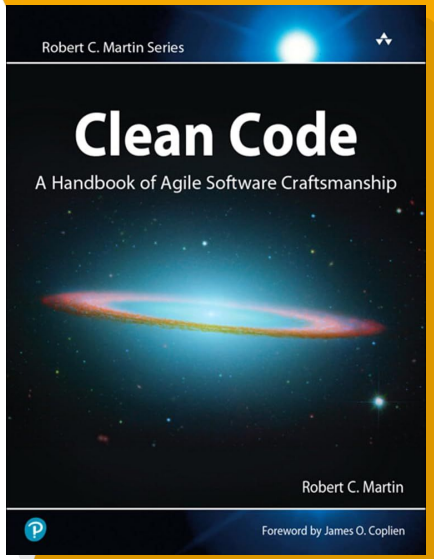
Herramientas

SonarQube, Jacoco...

sonarqube

Principios y Diseño

- **Principios SOLID**
- Patrones de diseño...



Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Calidad de código: complejidad ciclomática

OrderService.java

Complejidad: ALTA

```
public String processOrder(int orderType, boolean isPriority, double amount, String customerType)
{
String result = "";

 if (orderType == 1) {
     if (isPriority) {
         if (amount > 1000) {
           result = "Express processing for large priority order";
         } else {
           result = "Express processing for small priority order";
         }
     } else {
         if (amount > 500) {
           result = "Standard processing for large regular order";
         } else {
           result = "Standard processing for small regular order";
         }
     }
 } else if (orderType == 2) {
    switch (customerType) {
    case "VIP":
      if (amount > 2000) {
        [....]
 return result;
}
```

Calidad de código: complejidad ciclomática

OrderService.java

Complejidad: BAJA

```
public String processOrder(int orderType, boolean isPriority, double
amount, String customerType) {
 if (orderType == 1) {
   return processStandardOrder(isPriority, amount);
 }
 if (orderType == 2) {
   return processVipOrder(customerType, amount);
 }
 return processBulkOrder(amount);
}
```

Calidad de código: Cobertura

Mide el porcentaje de código fuente que se ejecuta durante las pruebas

¿Qué nos aporta?

- **Calidad** del test
- Identifica partes de **código sin probar** (identificamos escenarios por cubrir)

Enfoque Principal

NO hacemos test para buscar alta cobertura de código, sino para probar funcionalidad.

Una cobertura alta no garantiza calidad si los tests no validan los resultados correctamente. Es un indicador, no un objetivo.

SOLID

- **Single Responsibility:**
  - “Mis entidades deberían tener una y solo una razón de cambio”
- **Open/Closed:**
  - “Mis entidades deberían estar abiertas a extensión pero cerradas a modificación”
- **Liskov Substitution:**
  - “Las clases derivadas deben poder sustituir a su clase base sin alterar el correcto funcionamiento del sistema”
- **Interface Segregation:**
  - “Nunca se debería obligar a un cliente a implementar una interfaz que no utilice, las interfaces deberían ser específicas”
- **Dependency Inversion:**
  - “Las entidades deben depender de abstracciones”

TDD vs SOLID

	TDD	Principios SOLID
¿Qué son?	Metodología de desarrollo guiado por pruebas	Reglas sagradas e inquebrantables para escribir código mantenible.
Enfoque	¿Qué es lo mínimo que debo hacer para cumplir los requisitos?	¿Cómo evito que mi código sea un monstruo inmantenible?
¿Cuándo aplicarlos?	Cuando tengas requisitos concretos para implementar y quieras mejorar la calidad y documentación de tu código sin esfuerzo.	En las siguientes situaciones: 1. Siempre 2. Siempre 3. Por si acaso
Nivel de abstracción	Forma de desarrollar	Principios generales
¿Son obligatorios?	No, dependerá de la situación.	Si no los usáis, muere un gatito



06_

Kata: FizzBuzz 🎲

Kata: FizzBuzz 🎲

Dado un número entero:

- Si es múltiplo de 3, devuelve "Fizz".
- Si es múltiplo de 5, devuelve "Buzz".
- Si es múltiplo de 3 y 5, devuelve "FizzBuzz".
- En cualquier otro caso, devuelve el número como string.

https://github.com/AnaGciaSchz/tdd-workshop

Kata FizzBuzz: Test Cases

✅ `testNumberOne_ShouldReturnOne()` → `fizzBuzz(1) == "1"`

✅ `testNumberTwo_ShouldReturnTwo()` → `fizzBuzz(2) == "2"`

✅ `testNumberThree_ShouldReturnFizz()` → `fizzBuzz(3) == "Fizz"`

✅ `testNumberFive_ShouldReturnBuzz()` → `fizzBuzz(5) == "Buzz"`

✅ `testNumberSix_ShouldReturnFizz()` → `fizzBuzz(6) == "Fizz"` (múltiplo de 3)

✅ `testNumberTen_ShouldReturnBuzz()` → `fizzBuzz(10) == "Buzz"` (múltiplo de 5)

✅ `testNumberFifteen_ShouldReturnFizzBuzz()` → `fizzBuzz(15) == "FizzBuzz"` (múltiplo de 3 y 5)

✅ `testNumberThirty_ShouldReturnFizzBuzz()` → `fizzBuzz(30) == "FizzBuzz"`

07_

JUnit: Conceptos



JUnit

Conjunto de librerías que nos ayudan en el desarrollo de pruebas, mediante anotaciones dotamos de comportamiento a nuestra batería de pruebas.

@Test

Anotación principal que define un método como caso de prueba ejecutable.

@BeforeEach, @BeforeAll

Métodos de preparación ejecutados antes de cada test o de la suite completa.

@AfterEach, @AfterAll

Métodos de limpieza y liberación de recursos ejecutados tras las pruebas.

@Timeout, @Disabled

Anotaciones de control para tiempo máximo de ejecución y deshabilitación.

Guía oficial: https://junit.org/junit5/docs/current/user-guide/

Un framework de tests por lenguaje

JUNIT 5 · JAVA

```
@Test
assertEquals(esperado, real)
@ParameterizedTest
```

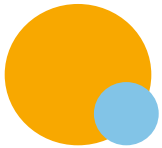
NUNIT · C# / .NET

```
[Test]
Assert.That(real,
Is.EqualTo(esperado))
[TestCase(...)]
```

PYTEST · PYTHON

```
def test_algo():
assert real == esperado
@pytest.mark.parametrize
```

Las ideas son las mismas en los tres: AAA, FIRST y Red-Green-Refactor. Solo cambia la sintaxis.



JUnit: @Test

Anotación que representa una función de test que se ejecutará

RandomTest.java

```
@Test  new *
void myRandomTest() {
    // Some test to be executed
}
```

JUnit: @BeforeEach, @BeforeAll

Funciones que se ejecutan antes de toda la suite de tests y antes de cada uno en particular

TestSetup.java

```java
@BeforeAll  new *
static void setUpEnvironment(){
    // global initialization before execution
}

@BeforeEach  new *
void setUpTest(){
    // test initialization stuff for test cases
}
```

JUnit: @AfterEach, @AfterAll

Funciones que se ejecutan después de toda la suite de tests y después de cada uno en particular

TestCleanup.java

```java
@AfterAll  new *
static void cleanEnvironment(){
    // global cleanup after all execution
}

@AfterEach  new *
void cleanTest(){
    // test cleanup for every test
}
```

JUnit: @Timeout

Configuración de parámetros de tiempo de espera y sus anotaciones equivalentes

Parameter value	Equivalent annotation
42	`@Timeout(42)`
42 ns	`@Timeout(value = 42, unit = NANOSECONDS)`
42 µs	`@Timeout(value = 42, unit = MICROSECONDS)`
42 ms	`@Timeout(value = 42, unit = MILLISECONDS)`
42 s	`@Timeout(value = 42, unit = SECONDS)`
42 m	`@Timeout(value = 42, unit = MINUTES)`
42 h	`@Timeout(value = 42, unit = HOURS)`
42 d	`@Timeout(value = 42, unit = DAYS)`

JUnit: assertTimeout()

Para validar el tiempo de ejecución de una función concreta

TimeoutTest.java

```java
@Test  new *
void checkTimeoutFunction() {
    School school = new School();

    assertTimeout(Duration.ofMillis(500), ()-> school.getDirection());
}
```

JUnit: @Disabled

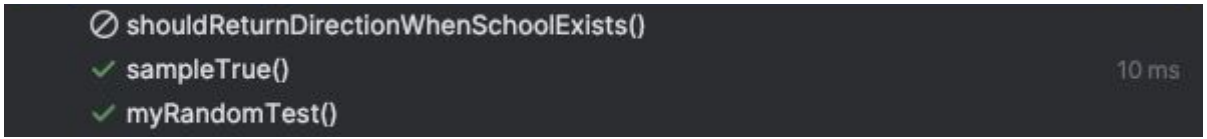
Para omitir temporalmente la ejecución de un test o de toda una suite de pruebas

DisabledTest.java

```java
@Disabled("my optional reason") new *
@Test
void shouldReturnDirectionWhenSchoolExists() {
    School school = new School();

    String direction = school.getDirection();

    assertEquals( expected: "Oviedo, Asturias", direction);
```



JUnit: assertThrows() / assertDoesNotThrow()

Para verificar que un bloque de código lanza (o no) una excepción esperada

ExceptionTest.java

```java
@Test  new *
void checkException() {
    assertThrows(NullPointerException.class, () -> {
        System.out.print("testing exceptions!");
        throw new NullPointerException();
    });
}
```

JUnit: assertEquals() / assertNotEquals()

Para comparar el valor esperado con el valor obtenido

EqualsTest.java

```java
@Test
void shouldReturnDirectionWhenSchoolExists() {
    School school = new School();

    String direction = school.getDirection();

    assertEquals( expected: "Oviedo, Asturias", direction);
```

JUnit: assertTrue() / assertFalse()

Para evaluar condiciones booleanas (verdadero o falso)

BooleanTest.java

```java
@Test  new *
void shouldReturnDirectionNotEmptyWhenSchoolExists() {
    School school = new School();

    String direction = school.getDirection();

    assertFalse(direction.isEmpty());
```

JUnit: @ParameterizedTest & @ValueSource

Nos permite ejecutar el mismo test con diferentes inputs

ValueSourceTest.java

```java
@ParameterizedTest(name = "{0} is a valid number")  new *
@ValueSource(strings = {"1", "2", "3"})
void checkNumbers(String textToCheck) {
    assertDoesNotThrow( () -> Integer.parseInt(textToCheck));
```





JUnit: @ParameterizedTest & @MethodSource

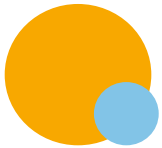
Fuente de inputs desde un *Factory method*

MethodSourceTest.java

```java
static Stream<Integer> myMethodSource() {
return Stream.of(1, 2, 3, 4, 5);
}

@ParameterizedTest
@MethodSource("myMethodSource")
void testSquareFunction(int input) {
int expected = input * input;
assertEquals(expected, square(input));
}

private int square(int n) {
return n * n;
}
```



Preparando el entorno 💻: dependencias

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.11.4</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-params</artifactId>
  <version>5.11.4</version>
  <scope>test</scope>
</dependency>
```



08_ SonarQube 🧪

SonarQube en local 🧪

SonarQube es una plataforma de análisis estático de código que detecta errores, vulnerabilidades y deuda técnica, ayudando a mantener código limpio y seguro.

Análisis Automático

Detecta bugs, code smells y vulnerabilidades.

Cobertura de Pruebas

Integra con JaCoCo para medir cobertura de tests.

Soporte Multi-Lenguaje

Compatible con Java, JavaScript, Python, etc.

Integración CI/CD

Funciona con Jenkins, GitHub Actions, GitLab CI/CD.

Dashboards Detallados

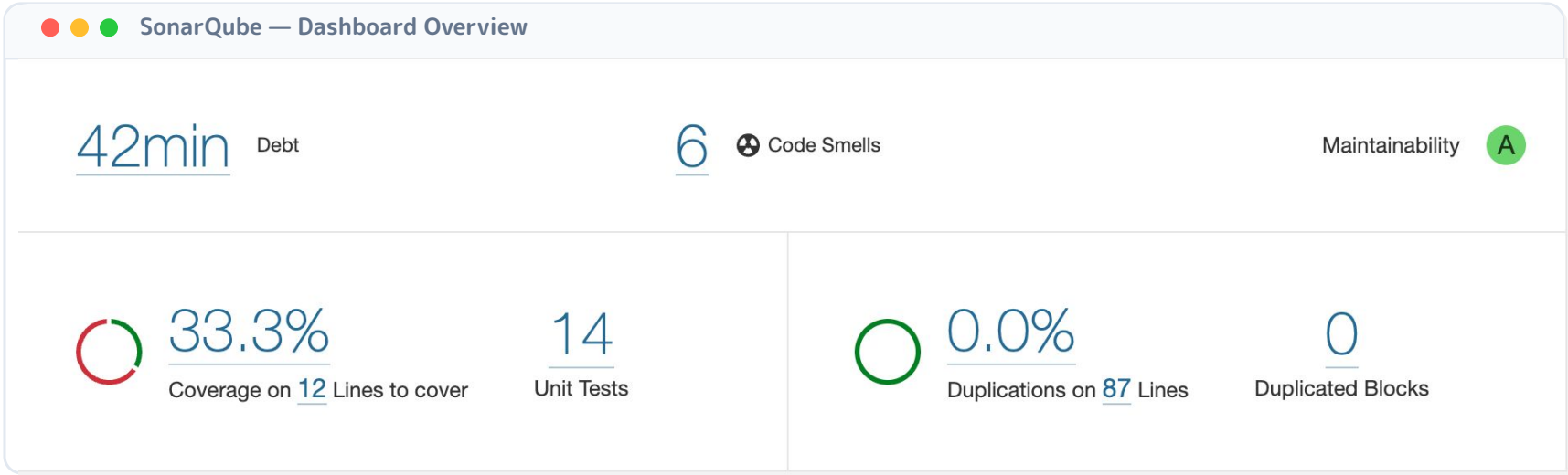
Métricas de calidad en una interfaz intuitiva.

Reglas Personalizables

Configura estándares de calidad según necesidades.

Calidad de código: SonarQube

Nos proporciona información muy interesante sobre nuestra base de código. Ej:



Antes de Sonar: Docker y JaCoCo

HERRAMIENTA 1

CONTENEDORES

Docker

- Empaqueta **una aplicación con todo lo que necesita** en un contenedor.
  - **Mismo resultado** en cualquier máquina
- En el taller levanta **SonarQube y PostgreSQL** sin instalarlos
- Un solo comando: **docker-compose up** y todo arranca

HERRAMIENTA 2

COBERTURA

JaCoCo

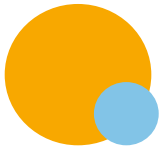
- ***Java Code Coverage***: mide qué líneas y ramas ejecutan tus tests
  - Genera un informe en target/site/jacoco
- SonarQube lo lee para **mostrar la cobertura en su dashboard**

SonarQube en local 🧪

Configurar **sonar-project.properties**

sonar-project.properties

```
sonar.projectKey=mi-proyecto
sonar.projectName=Mi Proyecto Java
sonar.host.url=http://localhost:9000
sonar.login=MI_TOKEN_DE_SONAR
sonar.language=java
sonar.sourceEncoding=UTF-8
sonar.tests=src/test/java
sonar.java.binaries=target/classes
sonar.test.inclusions=**/*Test.java
sonar.junit.reportPaths=target/surefire-reports
sonar.jacoco.reportPaths=target/jacoco.exec
```



SonarQube en local 🧪

1. Levantar con Docker una instancia de Sonar y de PostgreSQL

🌐 SonarQube en http://localhost:9000 | 🔑 Usuario y contraseña por defecto: `admin / admin`

docker-compose.yml

```
# Fichero docker-compose
url: https://github.com/miguelms95/tdd-tennis-workshop/blob/feat/sonar/docker-compose.yml

# Levantar los servicios:
$ docker-compose up
```

SonarQube en local 🧪

2. Generar un token en SonarQube

Paso 1: Acceso

- Ve a http://localhost:9000
- Inicia sesión con `admin / admin`

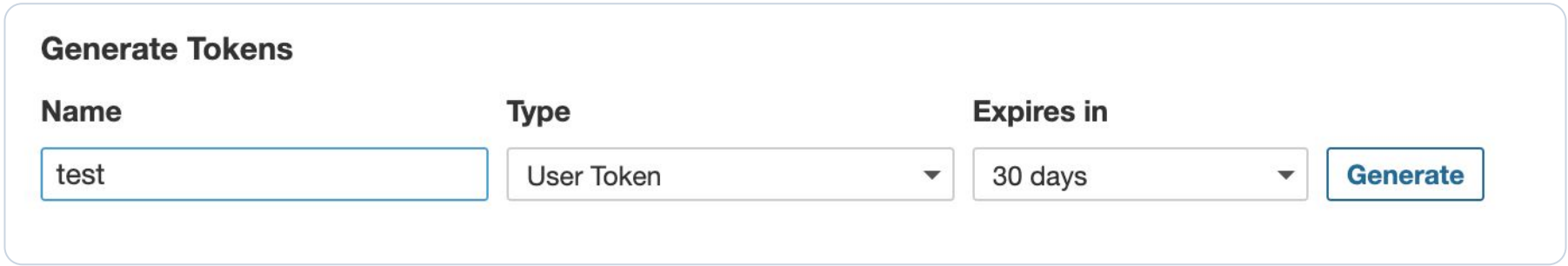
Paso 2: Crear Token

- En navegación: "My Account" → "Security"
  - Nombre: “test”
  - Tipo: “User token”

Paso 3: Guardar Token

- Copia el token generado *(lo usarás en el siguiente paso)*.





SonarQube en local 🧪

3. Pega el token en el fichero `sonar-project.properties`

sonar-project.properties

```
sonar.login=TU_TOKEN_SONAR
```

5. Añade las dependencias de Sonar y Jacoco al proyecto

Configura los plugins necesarios para capturar la cobertura de código.

🧪 **Sonar Plugin**
Análisis de código

📊 **Jacoco Plugin**
Reporte de cobertura

🎯 **Cobertura de Código**
Métricas automáticas en Sonar

SonarQube en local 🧪

```
<build>
   <plugins>
       <plugin>
           <groupId>org.apache.maven.plugins</groupId>
           <artifactId>maven-surefire-plugin</artifactId>
           <version>3.5.2</version>
       </plugin>
       <plugin>
           <groupId>org.sonarsource.scanner.maven</groupId>
           <artifactId>sonar-maven-plugin</artifactId>
           <version>3.9.1.2184</version>
       </plugin>
       <plugin>
           <groupId>org.jacoco</groupId>
           <artifactId>jacoco-maven-plugin</artifactId>
           <version>0.8.10</version>
           <executions>
               <execution>
                   <goals>
                       <goal>prepare-agent</goal>
                   </goals>
               </execution>
               <execution>
                   <id>report</id>
                   <phase>verify</phase>
                   <goals>
                       <goal>report</goal>
```

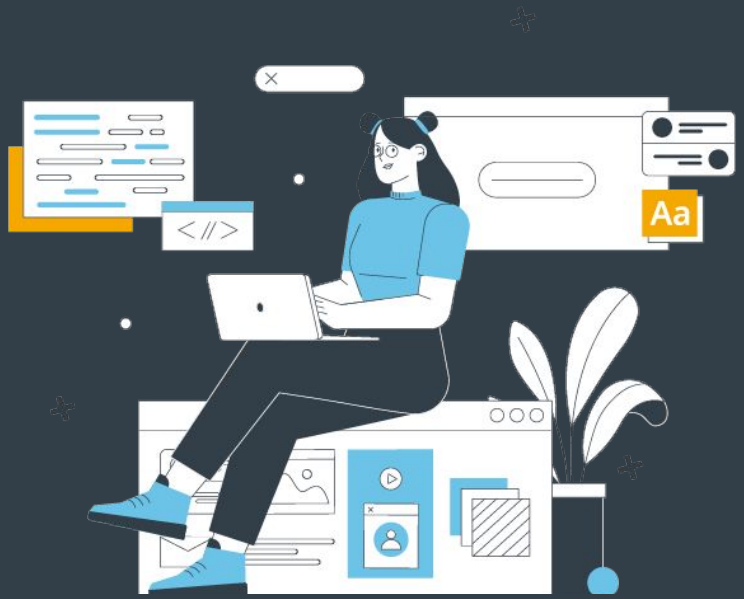
SonarQube en local 🧪

bash - ejecutar análisis SonarQube

```
mvn clean verify sonar:sonar -Dsonar.login=TU_TOKEN_AQUI
```

📊 **Reporte de Jacoco**
`target/site/jacoco/index.html`

🌐 **Dashboard de SonarQube**
http://localhost:9000



09_

Kata: tenis 🎾

Kata: tennis 🎾

- Puntuación de cada juego: 0, 15, 30, 40
- Si los dos jugadores tienen 40 puntos están en “deuce”
  - Si un jugador hace punto, estará en “ventaja”
  - Si el jugador con ventaja hace punto, gana el juego
  - Si el jugador sin ventaja hace punto, vuelven a “deuce”
- La nomenclatura de los puntos: “love”, “fifteen”, “thirty” y “forty”.
- El juego lo gana el primero que llega a 4 puntos con al menos 2 de ventaja

https://github.com/AnaGciaSchz/tdd-workshop



10_

Mockito



Test doubles 🎭

- Objetos que **imitan el comportamiento** de dependencias reales.
- **Facilitan** pruebas unitarias aisladas.
- **Evitan dependencias** externas como bases de datos o servicios HTTP.
- 📌 Ejemplo: Un test de una clase OrderService no debería depender de una base de datos real para verificar su funcionalidad.

Test doubles 🎭: tipos

- **Dummy**: Se usan solo para cumplir con la firma del método, pero no tienen comportamiento.
- **Stub**: Configura el comportamiento. Devuelve valores predefinidos en llamadas específicas.
- **Spy**: Registran llamadas para ser verificadas después.
- **Mock**: Permiten definir expectativas y verificar interacciones.
- **Fake**: Implementaciones simples y funcionales para pruebas.

🎭 Mockito

Mockito es una de las soluciones más adoptadas en el ecosistema Java.

🌐 **site.mockito.org**



🎭 Mockito: añadir al pom.xml

- dependencias:

```
<dependency>
   <groupId>org.mockito</groupId>
   <artifactId>mockito-core</artifactId>
   <version>5.15.2</version>
   <scope>test</scope>
</dependency>
<dependency>
   <groupId>org.mockito</groupId>
   <artifactId>mockito-junit-jupiter</artifactId>
   <version>5.10.0</version>
   <scope>test</scope>
</dependency>
```

🎭 Mockito - Creación de mocks

Mockito permite simular objetos dependientes en los test:

```
HawkEye dummyHawkEye = mock(HawkEye.class);
```

- **mock(Class<T> classToMock)**

  Crea e inicializa una instancia simulada (mock) de la clase o interfaz especificada.

🎭 Mockito - Configuración de Comportamiento (Stubbing)

Simula respuestas de métodos específicos de los mocks.

- **when(...).thenReturn(...) → Devuelve un valor predefinido.**

```
when(hawkEye.isBallIn()).thenReturn(false));
```

- **when(...).thenThrow(...) → Lanza una excepción en ciertas condiciones.**

```
when(hawkEye.isBallIn()).thenThrow(new
HawkEyeUnavailableException("Ojo de halcón sin
conexión"));
```

🎭 Mockito - Verificación de interacción (1)

Comprueba si se han llamado métodos específicos en un mock.

- **verify(mock).method(args)** → Verifica que el método fue llamado.
- **verify(mock, times(n))** → Verifica el número exacto de veces.

```
// Ejemplo: verificar llamada a un método específico
verify(scoreboard).display("fifteen-love");
```

🎭 Mockito - Verificación de interacción (2)

Ejemplo con el número de llamadas

```
verify(scoreboard, times(3)).display(anyString());
```

- **never()**

Nunca debe llamarse el método.

- **atLeast(n)**

Se llama al menos n veces.

- **atMost(n)**

Se llama como máximo n veces.

🎭 Mockito - Spy

Permite usar el comportamiento real del objeto y modificar ciertas llamadas.

- **No altera el comportamiento real del objeto espiado**

```
List<String> list = spy(new ArrayList<>());
list.add("Hello");
verify(list).add("Hello");
assertEquals(1, list.size());
```

🎭 Mockito - Spy vs Mock

🔹 Mock altera el comportamiento real del objeto

La lista no guarda los elementos añadidos y mantiene su tamaño en 0.

```
List<String> mockedList = mock(ArrayList.class);
mockedList.add("Hello");
mockedList.add("World!");
verify(mockedList).add("Hello");
verify(mockedList).add("World!");
assertEquals(0, mockedList.size());
```

🎭 Mockito: stub

Stub (estado): el ojo de halcón dice que entra y comprobamos el marcador resultante

- **Devuelve respuestas predefinidas para facilitar el test**

```
@Test  Ana
@DisplayName("Stub (estado): el ojo de halcón dice que entra y
void stubChecksResultingState() {
    // Arrange
    when(hawkEye.isBallIn()).thenReturn( value: true);

    // Act
    match.resolvePlayerOneShot();

    // Assert
    assertEquals( expected: "fifteen-love", match.score());
}
```

🎭 Mockito: mock

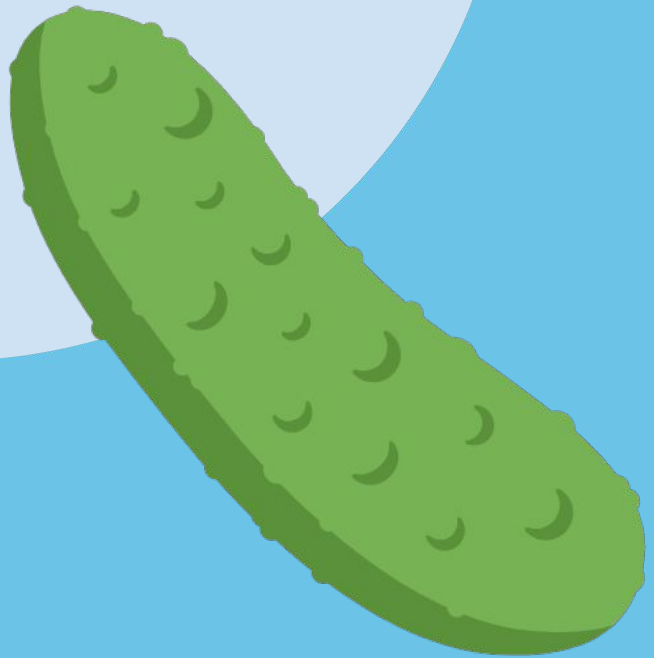
Mock (comportamiento): el ojo de halcón dice que entra y comprobamos que el partido envió el marcador al panel

- **Verifica comportamientos e interacciones**

```
@Test  Ana
@DisplayName("Mock (comportamiento): el ojo de halcón dice que entra
        "marcador al panel")
void mockChecksInteractionWithDependency() {
    // Arrange
    when(hawkEye.isBallIn()).thenReturn( value: true);

    // Act
    match.resolvePlayerOneShot();

    // Assert
    verify(scoreboard).display( score: "fifteen-love");
}
```



11_

Cucumber & TestContainers

¿Qué es Cucumber?

LA IDEA

Escribimos los requisitos en lenguaje natural (Gherkin) y Cucumber los ejecuta como tests.

POR QUÉ

- Negocio, QA y desarrollo leen el mismo documento
- El requisito y el test no se desincronizan
- Obliga a pensar ejemplos concretos antes de programar

EN EL TALLER

Describimos el marcador de un juego de tenis y comprobamos que la kata lo cumple.

OJO

“Cucumber no arregla un requisito mal entendido: solo lo hace visible.”

Gherkin con la kata de tenis

`src/test/resources/features/tennis.feature`

```
Feature: Marcador de un juego de tenis

    Scenario: Empieza el juego
        Given un juego nuevo
       Then el marcador es "Love-Love"

    Scenario: Ventaja tras deuce
        Given un juego nuevo
       When el jugador 1 gana 3 puntos
        And el jugador 2 gana 3 puntos
        And el jugador 1 gana 1 punto
      Then el marcador es "Advantage Player 1"
```

CÓMO SE LEE

Feature
La funcionalidad
Scenario
Un caso concreto
Given
Estado inicial (Arrange)
When
La acción (Act)
Then
El resultado esperado (Assert)
And
Repite el paso anterior

Scenario Outline: la tabla de la kata

src/test/resources/features/tennis.feature

```
Scenario Outline: Marcadores del juego
        Given un juego nuevo
   When el jugador 1 gana <p1> puntos
    And el jugador 2 gana <p2> puntos
   Then el marcador es "<marcador>"

              Examples:
    | p1 | p2 | marcador           |
    | 0  | 0  | Love-Love          |
    | 1  | 0  | Fifteen-Love       |
    | 3  | 3  | Deuce              |
    | 4  | 3  | Advantage Player 1 |
    | 5  | 3  | Player 1 wins      |
```

QUÉ ES

Un escenario plantilla que se ejecuta una vez por cada fila de Examples.

CÓMO FUNCIONA

Cada <p1>, <p2> y <marcador> se sustituye por el valor de su columna.

TE SUENA

Es el equivalente a @ParameterizedTest en JUnit, con la misma tabla de casos de la kata.

De frase a código: los steps

TennisSteps.java

```java
public class TennisSteps {
  private TennisGame game;

  @Given("un juego nuevo")
public void unJuegoNuevo() {
game = new TennisGame("player1", "player2");
}

@When("el jugador {int} gana {int} punto(s)")
public void ganaPuntos(int jugador, int puntos) {
for (int i = 0; i < puntos; i++)
game.wonPoint("player" + jugador);
}

@Then("el marcador es {string}")
public void elMarcadorEs(String esperado) {
assertEquals(esperado, game.getScore());
}
}
```

CÓMO SE CONECTA

- Cada frase busca el método con la misma expresión
- {int} y {string} llegan como parámetros
- punto(s) es texto opcional: vale para «1 punto» y «3 puntos»
- Si una frase no tiene step, Cucumber la marca como undefined y te propone el esqueleto vacío

FÍJATE

El step solo traduce: la lógica sigue en el código de la app.

Runner y dependencias

RunCucumberTest.java

```java
@Suite
@IncludeEngines("cucumber")
@SelectClasspathResource("features")
public class RunCucumberTest {
}
```

La clase está vacía: las anotaciones le dicen a JUnit que use el motor de Cucumber y busque los .feature en src/test/resources/features.

DEPENDENCIAS (TEST)

- cucumber-java
- cucumber-junit-platform-engine
- junit-platform-suite

AL EJECUTAR mvn test

Surefire → JUnit Platform → @Suite → Cucumber → features → steps

Cucumber frente a JUnit

JUNIT: TESTS UNITARIOS

- Prueban una clase o un método
- Los lee el equipo de desarrollo
- Muy rápidos: se escriben muchos
- Ejemplo: los tests de TDD que ya tenemos

CUCUMBER: TESTS FUNCIONALES

- Prueban un comportamiento que el negocio reconoce
- Los puede leer cualquiera del equipo
- Más lentos: pocos escenarios clave
- En la kata: tennis.feature

NO SE SUSTITUYEN

Cucumber dice qué debe pasar; JUnit cubre los detalles.

Alternativas a los dobles de prueba (Test Doubles como Mockito)

BD EMBEBIDA

H2 o SQLite en memoria.

- Rápida, no hay que instalar nada
- No es tu BD de producción: el SQL puede comportarse distinto

DOCKER A MANO

docker-compose con la BD real.

- Misma BD que producción
- Hay que levantarla antes y limpiar los datos entre tests

TESTCONTAINERS

La BD real en Docker, gestionada desde el propio test.

- Arranca y se destruye sola
- Necesita Docker en la máquina

Testcontainers con PostgreSQL

Ejemplo: si la kata guardara los partidos

```
@Testcontainers
class MatchRepositoryIT {

@Container
static PostgreSQLContainer<?> postgres =
  new PostgreSQLContainer<>("postgres:16");

@Test
void guardaElResultadoDeUnPartido() {
// Arrange
var repo = new MatchRepository(
postgres.getJdbcUrl(),
postgres.getUsername(),
postgres.getPassword());
// Act
repo.save("Player 1", "Player 2", "Player 1 wins");
// Assert
assertEquals(1, repo.count());
}
}
```

QUÉ HACE

- @Container arranca un PostgreSQL real en Docker
- El test usa su URL, usuario y contraseña
- Al terminar, el contenedor se destruye

QUÉ NECESITAS

- Docker arrancado
- Testcontainers y el driver de PostgreSQL (scope test)

CUÁNDO

Tests de integración (mvn verify), no unitarios.

Mensajes clave

CUCUMBER

- Requisitos en lenguaje natural que se ejecutan
- Given/When/Then es AAA con otras palabras
- Scenario Outline es un test parametrizado
- Complementa a los tests unitarios, no los sustituye

TESTCONTAINERS

- Infraestructura real en los tests, sin instalar nada
- Para integración, no para unitarios

REGLA DE ORO

“Los unit tests protegen el código. Los funcionales protegen lo que el usuario espera.”



12_

ENTREGABLE

Kata: Store 📦

Kata: Registro de Pedidos de Tienda 📦

- Queremos desarrollar una **tienda** que procese pedidos de usuarios.
- Usuarios
  - Id, nombre y apellidos
- Pedidos
  - Id de pedido, id de usuario, estado y precio
  - Estado:
    - Draft, Processed, Dispatched, Delivered

https://github.com/AnaGciaSchz/tdd-workshop



Tarea evaluable

QUÉ ENTREGAR

1. Un registro de pedidos sencillo desarrollado con TDD:
   - Las pruebas se escriben antes que el código
   - Se recorre el ciclo Red-Green-Refactor
   - Pruebas unitarias
   - Al menos una prueba de integración o end-to-end
   - Todo en verde después de refactorizar
2. Analizar el proyecto con SonarQube en local y tener al menos un 80 % de cobertura (incluir una captura de pantalla en el README del proyecto)
3. Hacer tests con JUnit 5 y Mockito
4. Respetar los principios que hemos dado para hacer buenos tests
5. [OPCIONAL] Incluir ejemplos de cómo haríais un test de Cucumber y un test con Testcontainers

CÓMO SE EVALÚA

- Entrega por correo
- Supone el 100% de la nota
- Nota mínima para aprobar: 5
- Nota máxima: 10 (La parte opcional os suma 1,5 puntos a lo que tengáis)

NECESARIO

- Haz un commit en cada fase (rojo, verde, refactor) para que el ciclo se vea en el historial. Con hacerlo al principio del desarrollo, a la mitad y luego al final sirve.
- Entregar un proyecto que compile
- No entregar con tests en rojo

Mandad un correo con el link a vuestra entrega en GitHub a ana.garcia@nextdigital.es

Gracias!

Linkt.ee

La simplicidad es la máxima sofisticación
Leonardo Da Vinci

